

Lottery Scheduler in xv6 Final Project Report

Sasha Delgado

CIS 345-Operating Systems

05/13/2026

This project required me to modify the xv6 operating system to create a Lottery Scheduler. A lottery scheduler works by giving processes tickets. It randomly picked a ticker during each scheduling cycle and the process holding the winning ticket gets CPU time. The lesson of this project was to understand how scheduling works inside an operating system kernel and how the processes compete for CPU time.

Learning Objectives:

- How xv6 is structured
- How CPU scheduling works
- How to modify the xv6 kernel
- How to add new system calls
- How to test and analyze scheduler performance

The following files have been Modified:

- kernel/proc.h
- kernel/proc.c
- kernel/sysproc.c
- kernel/syscall.h
- kernel/syscall.c
- user/user.h
- user/usys.pl
- Makefile
- kernel/pstat.h
- user/lotterytest.c

Implementation Steps

1. Added tickets and ticks to the proc structure.

```
vboxuser@VM1: ~/xv6-riscv — nano kernel/proc.h
GNU nano 8.4 kernel/proc.h
/* 256 */ uint64 t3;
/* 264 */ uint64 t4;
/* 272 */ uint64 t5;
/* 280 */ uint64 t6;
};

enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state; // Process state
    void *chan; // If non-zero, sleeping on chan
    int killed; // If non-zero, have been killed
    int xstate; // Exit status to be returned to parent's wait
    int pid; // Process ID
    int tickets; // number of Lottery tickets
    int ticks; // number of times scheduled

    // wait_lock must be held when using this:
    struct proc *parent; // Parent process

    // these are private to the process, so p->lock need not be held.
    uint64 kstack; // Virtual address of kernel stack
    uint64 sz; // Size of process memory (bytes)
    pagetable_t pagetable; // User page table
    struct trapframe *trapframe; // data page for trampoline.S
    struct context context; // switch() here to run process
    struct file *ofile[NFILE]; // Open files
    struct inode *cwd; // Current directory
    char name[16]; // Process name (debugging)
};
```

2. Set default ticket values inside allocproc().

```
vboxuser@VM1: ~/xv6-riscv — nano kernel/proc.c
GNU nano 8.4 kernel/proc.c
// If there are no free procs, or a memory allocation fails, return 0.
static struct proc*
allocproc(void)
{
    struct proc *p;

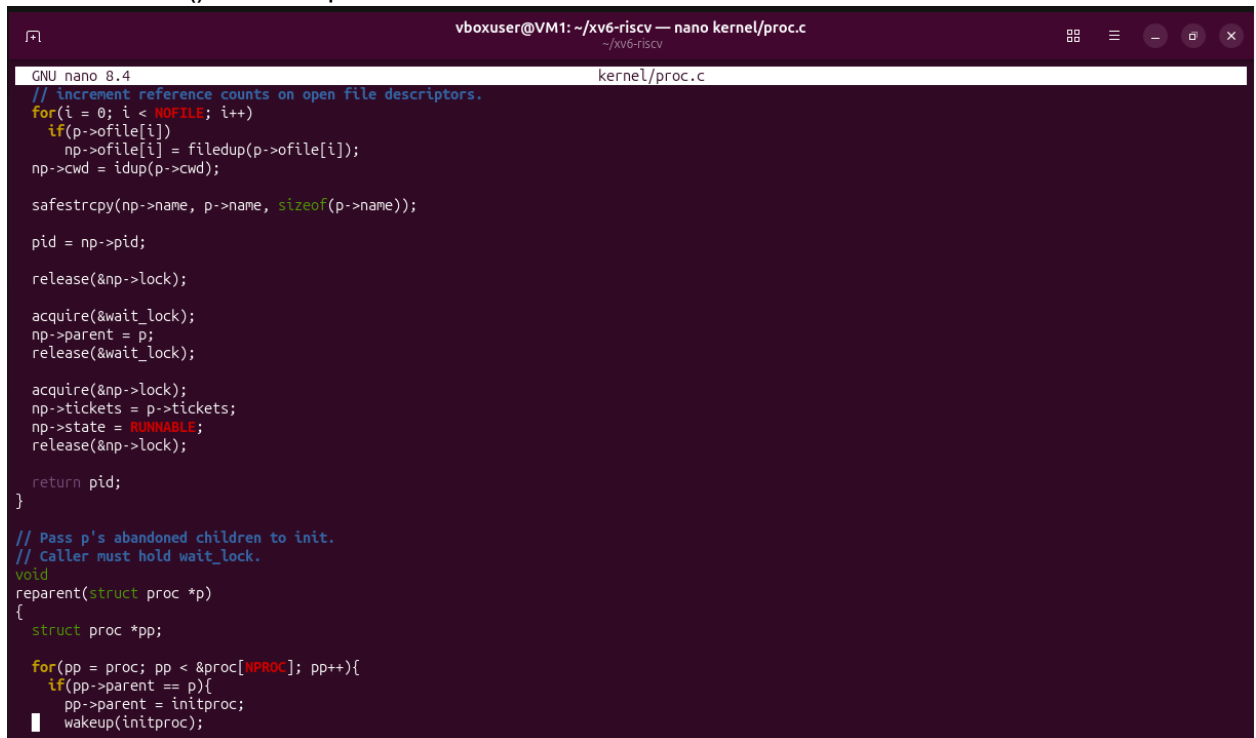
    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->state == UNUSED) {
            goto found;
        } else {
            release(&p->lock);
        }
    }
    return 0;

found:
    p->pid = allocpid();
    p->state = USED;
    p->tickets = 1;
    p->ticks = 0;

    // Allocate a trapframe page.
    if((p->trapframe = (struct trapframe *)kalloc()) == 0){
        freeproc(p);
        release(&p->lock);
        return 0;
    }

    // An empty user page table.
    p->pagetable = proc_pagetable(p);
    if(p->pagetable == 0){
        freeproc(p);
        release(&p->lock);
        return 0;
    }
}
```

3. Modified fork() so child processes inherit tickets.



The screenshot shows a terminal window with the title bar "vboxuser@VM1: ~/xv6-riscv — nano kernel/proc.c". The editor is GNU nano 8.4, and the file is kernel/proc.c. The code is as follows:

```
// increment reference counts on open file descriptors.
for(i = 0; i < NOFILE; i++)
    if(p->ofile[i])
        np->ofile[i] = filedup(p->ofile[i]);
np->cwd = idup(p->cwd);

safestrncpy(np->name, p->name, sizeof(p->name));

pid = np->pid;

release(&np->lock);

acquire(&wait_lock);
np->parent = p;
release(&wait_lock);

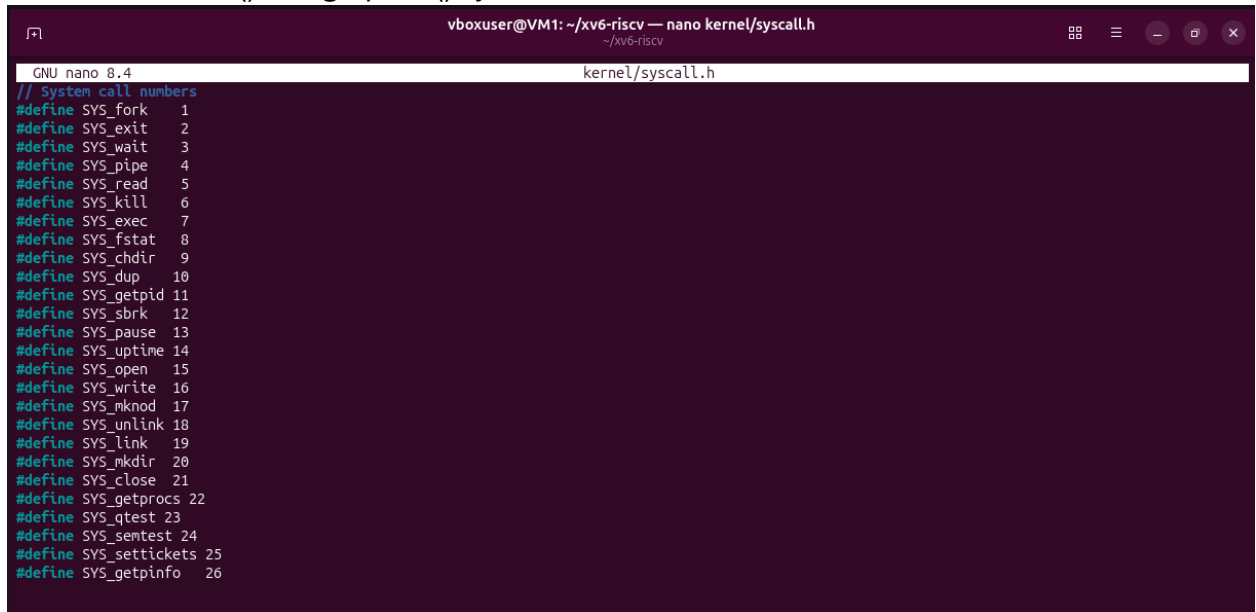
acquire(&np->lock);
np->tickets = p->tickets;
np->state = RUNNABLE;
release(&np->lock);

return pid;
}

// Pass p's abandoned children to init.
// Caller must hold wait_lock.
void
reparent(struct proc *p)
{
    struct proc *pp;

    for(pp = proc; pp < &proc[NPROC]; pp++){
        if(pp->parent == p){
            pp->parent = initproc;
            wakeup(initproc);
        }
    }
}
```

4. Added settickets() and getpinfo() system calls.



The screenshot shows a terminal window with the title bar "vboxuser@VM1: ~/xv6-riscv — nano kernel/syscall.h". The editor is GNU nano 8.4, and the file is kernel/syscall.h. The code is as follows:

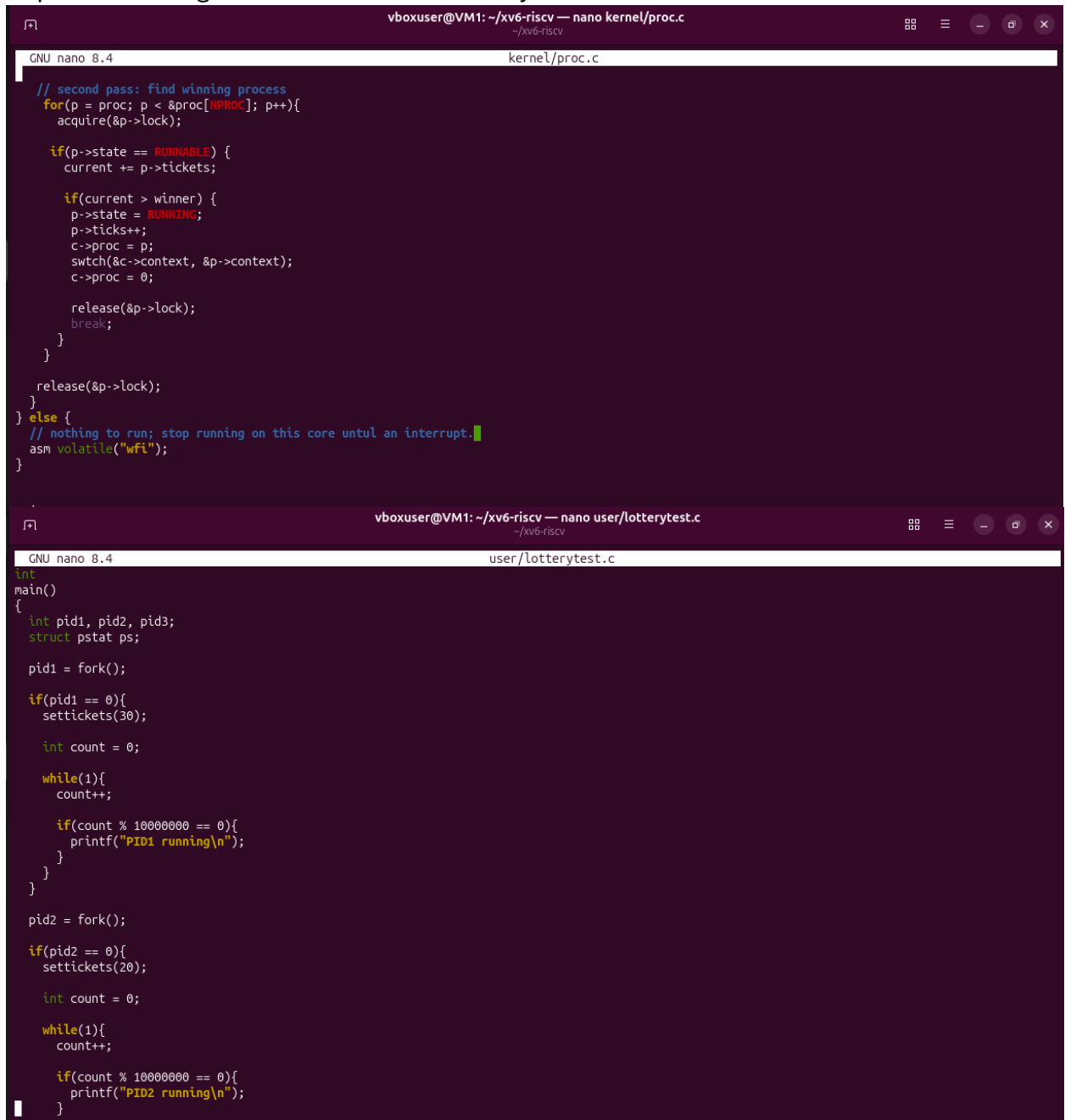
```
// System call numbers
#define SYS_fork 1
#define SYS_exit 2
#define SYS_wait 3
#define SYS_pipe 4
#define SYS_read 5
#define SYS_kill 6
#define SYS_exec 7
#define SYS_fstat 8
#define SYS_chdir 9
#define SYS_dup 10
#define SYS_getpid 11
#define SYS_sbrk 12
#define SYS_pause 13
#define SYS_uptime 14
#define SYS_open 15
#define SYS_write 16
#define SYS_mknod 17
#define SYS_unlink 18
#define SYS_link 19
#define SYS_mkdir 20
#define SYS_close 21
#define SYS_getprocs 22
#define SYS_qtest 23
#define SYS_semtest 24
#define SYS_settickets 25
#define SYS_getpinfo 26
```

```
vboxuser@VM1: ~/xv6-riscv — nano kernel/syscall.c
GNU nano 8.4 kernel/syscall.c
// Prototypes for the functions that handle system calls.
extern uint64 sys_fork(void);
extern uint64 sys_exit(void);
extern uint64 sys_wait(void);
extern uint64 sys_pipe(void);
extern uint64 sys_read(void);
extern uint64 sys_kill(void);
extern uint64 sys_exec(void);
extern uint64 sys_fstat(void);
extern uint64 sys_chdir(void);
extern uint64 sys_dup(void);
extern uint64 sys_getpid(void);
extern uint64 sys_sbrk(void);
extern uint64 sys_pause(void);
extern uint64 sys_uptime(void);
extern uint64 sys_open(void);
extern uint64 sys_write(void);
extern uint64 sys_mknod(void);
extern uint64 sys_unlink(void);
extern uint64 sys_link(void);
extern uint64 sys_mkdir(void);
extern uint64 sys_close(void);
extern uint64 sys_getprocs(void);
extern uint64 sys_qtest(void);
extern uint64 sys_semtest(void);
extern uint64 sys_settickets(void);
extern uint64 sys_getpinf(void);

vboxuser@VM1: ~/xv6-riscv — nano kernel/syscall.c
GNU nano 8.4 kernel/syscall.c
// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
[SYS_fork] sys_fork,
[SYS_exit] sys_exit,
[SYS_wait] sys_wait,
[SYS_pipe] sys_pipe,
[SYS_read] sys_read,
[SYS_kill] sys_kill,
[SYS_exec] sys_exec,
[SYS_fstat] sys_fstat,
[SYS_chdir] sys_chdir,
[SYS_dup] sys_dup,
[SYS_getpid] sys_getpid,
[SYS_sbrk] sys_sbrk,
[SYS_pause] sys_pause,
[SYS_uptime] sys_uptime,
[SYS_open] sys_open,
[SYS_write] sys_write,
[SYS_mknod] sys_mknod,
[SYS_unlink] sys_unlink,
[SYS_link] sys_link,
[SYS_mkdir] sys_mkdir,
[SYS_close] sys_close,
[SYS_getprocs] sys_getprocs,
[SYS_qtest] sys_qtest,
[SYS_semtest] sys_semtest,
[SYS_settickets] sys_settickets,
[SYS_getpinf] sys_getpinf,
};

void
syscall(void)
{
    int num;
```

5. Replaced the original scheduler with a lottery scheduler.



```
vboxuser@VM1: ~/xv6-riscv — nano kernel/proc.c
GNU nano 8.4 kernel/proc.c

// second pass: find winning process
for(p = proc; p < &proc[NPROC]; p++){
    acquire(&p->lock);

    if(p->state == RUNNABLE) {
        current += p->tickets;

        if(current > winner) {
            p->state = RUNNING;
            p->ticks++;
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;

            release(&p->lock);
            break;
        }
    }

    release(&p->lock);
}
} else {
    // nothing to run; stop running on this core until an interrupt.
    asm volatile("wfi");
}
}

vboxuser@VM1: ~/xv6-riscv — nano user/lotterytest.c
GNU nano 8.4 user/lotterytest.c

int
main()
{
    int pid1, pid2, pid3;
    struct pstat ps;

    pid1 = fork();

    if(pid1 == 0){
        settickets(30);

        int count = 0;

        while(1){
            count++;

            if(count % 10000000 == 0){
                printf("PID1 running\n");
            }
        }
    }

    pid2 = fork();

    if(pid2 == 0){
        settickets(20);

        int count = 0;

        while(1){
            count++;

            if(count % 10000000 == 0){
                printf("PID2 running\n");
            }
        }
    }
}
```

```
vboxuser@VM1: ~/xv6-riscv — nano user/lotterytest.c
GNU nano 8.4 user/lotterytest.c
}
}

pid3 = fork();

if(pid3 == 0){
    settickets(10);

    int count = 0;

    while(1){
        count++;

        if(count % 10000000 == 0){
            printf("PID3 running\n");
        }
    }
}

for(volatile int i = 0; i < 5000000000; i++){
    ;
}

getpinfo(&ps);

printf("PID\ttTickets\ttTicks\n");

for(int i = 0; i < NPROC; i++){
    if(ps.inuse[i]){
        printf("%d\t%d\t%d\n",
            ps.pid[i],
            ps.tickets[i],
            ps.ticks[i]);
    }
}

kill(pid1);
kill(pid2);
kill(pid3);

wait(0);
wait(0);
wait(0);

exit(0);
}

vboxuser@VM1:~/xv6-riscv$ make clean
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*/*.o */*.d */*.asm */*.sym \
kernel/kernel fs.img \
mkfs/mkfs .gdbinit \
    user/usys.S \
user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests us
er/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_getprocs user/_qtest user/_semtest user/_lotterytest
```

[illegible]

```

riscv64-linux-gnu-objdump -S user/_semtest > user/semtest.asm
riscv64-linux-gnu-objdump -t user/_semtest | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/semtest.sym
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -march=rv64gc -mabi=lp64d -fcommon -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -fno-stack-protector -fno-pie -no-pie -c -o user/lotterytest.o user/lotterytest.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_lotterytest user/lotterytest.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_lotterytest > user/lotterytest.asm
riscv64-linux-gnu-objdump -t user/_lotterytest | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/lotterytest.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_str
essfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_getprocs user/_qtest user/_semtest user/_lot
terytest
mmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
balloc: first 1089 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img
,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

```

```

vboxuser@VM1: ~/xv6-riscv — make qemu
~/xv6-riscv

hart 1 starting
hart 2 starting
init: starting sh
$ lotterytest
PID03 running
PPID02 running
D1 running
PPID1 running
PID1 running
PID1 running
ID2 running
ID3 running
PPID1 running
PID2 running
PID1 running
PID2 running
ID3 running
PPID3 running
ID2 running
PPID1 running
PID1 running
PID2 running
PID2 running
PID2 running
PID1 running
PID1 running
PID2 running
ID3 running
PPID1 running
PID2 running
PIDID3 running
PID2 running
PPID2 running
1 running
PID2 running
PID1 ruPID3 running
nning
ID2 running

```


PID1 running
PID2 running
PID3 running
PPID2 running
PID1 running
PPID1 running
ID2 running
PID1 running
PID2 running
PID1 running
PID3 running
PID3 running
PIPID2 running
ID1 running
PID2 running
PPID2 running
PID2 running
ID1 running
PID2 running
D3 running
PID1 running
PID2 running
PID1 running
PID3 running
PID2 running
PPID1 running
ID2 running
PID1 running
PID2 running
PID3 running
PID1 running
PID2 running
PID1 running
PID3 running
PID2 running
PID3 running
PID1 running
PID2 running
PID3 running

PID1 running
PID2 running
PPID3 running
PID2 running
PID3 running
ID1 running
PID2 running
PID3 running
PID1 running
PID2 running
PID3 running
PID1 running
PID2 running
PID3 rPID1 running
PID2 running
PID2 running
PID1 running
PPID1 running
unning
PID1 running
ID2 running
PPID3 running
PID2 running
PID3 running
PID2 running
ID1 running
PID3 running
PID2 running
PID1 running
PID3 running
PID2 running
PPID3 running
PID2 running
ID1 running
PID2 PID3 running

```
PID1 running
running
PPID1 running
PID1 running
PPID3 running
PID3 running
ID1 running
PPID3 running
PID3 runID2 running
ID1 running
PID1 running
PID1 running
ning
PID1 running
PID2 running
PID3 running
PID1 ruPID2 running
PID3 running
PPnning
ID2 running
PID2 running
PPID1 running
ID3 running
PID1 running
ID2 running
PID3 running
PID2 running
PID1 running
PID2 runnPID1 running
ing
PID1 running
PID2 running
PID3 running
PID1 running
PID2 running
PID3 running
PID1 running
PID1 running
PID3 running
PID2 running
PID1 running
PPID1 PrIInning
D2 running
PID1 running
PID2 running
PD3 runnID1 rPuInPning
D2 running
PID2 PID1 running
running
PID1 running
PPID1 running
ning
ID3 running
PPID2 running
PPID1 running
ID3 running
ID2 running
PID1 running
PID3 PIDP1 IDrunni2ng
ruPnnID1i rIungnr
PID2 running
PnInxec failed
$ ing
PID3 running
PPI2 DrIun1 niruDnn3g
ning
PPID2 running
ID1P Ir D2unning
running
```

```

PID1 running
running
PID2 running
PPID2 running
PIDPID3PID12 ru running
PID 20n nriunrning
PnIgDnni1
running
PID2 running
PPID1nI rg
Tickets Ticks
1 1 53
2 1 14
3 1 42
4 30 71
5 20 uD2P Irrnuing
D3 running
nning
PID1 runPID2niP ng
rPID1 IruDu3 nnnnirinnugg
PID1 running
P
ID1 nruPID2 running
PnIDn1n2g r
nPuIDn1 runnniSiinnng
gP
g
PID3 IruD1 rnuPInnninDg
9
6 10 63
2i$

```

Testing

I created a user program called lotterytest.c. That program created three child processes with different ticket counts and prints scheduler statistics.

Ticket Values:

PID4=30 tickets

PID5=20 tickets

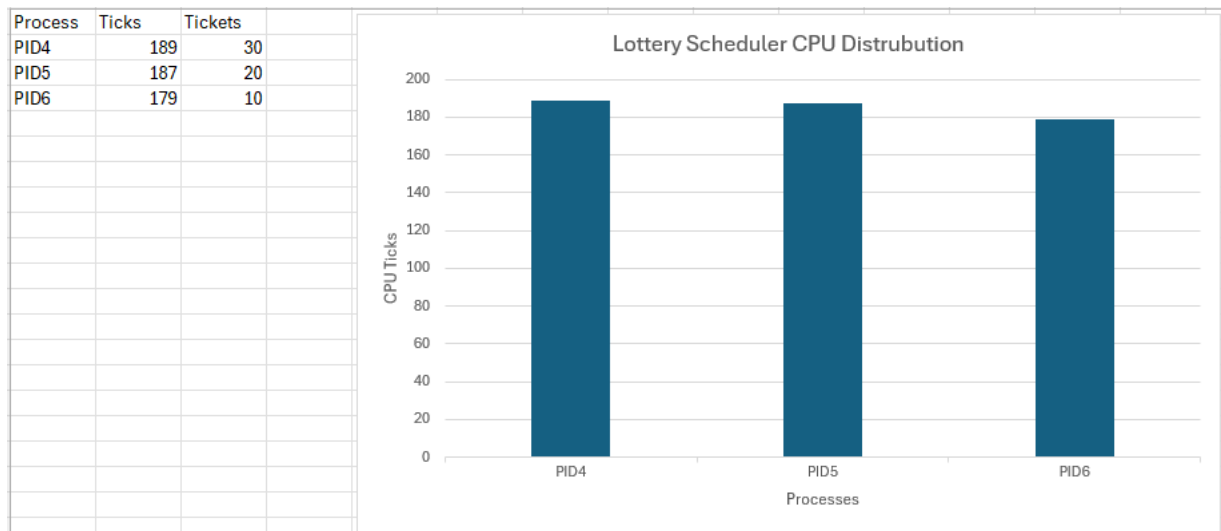
PID6=10 tickets

Scheduler Output:

PID	Tickets	Ticks
4	30	189
5	20	187
6	10	179

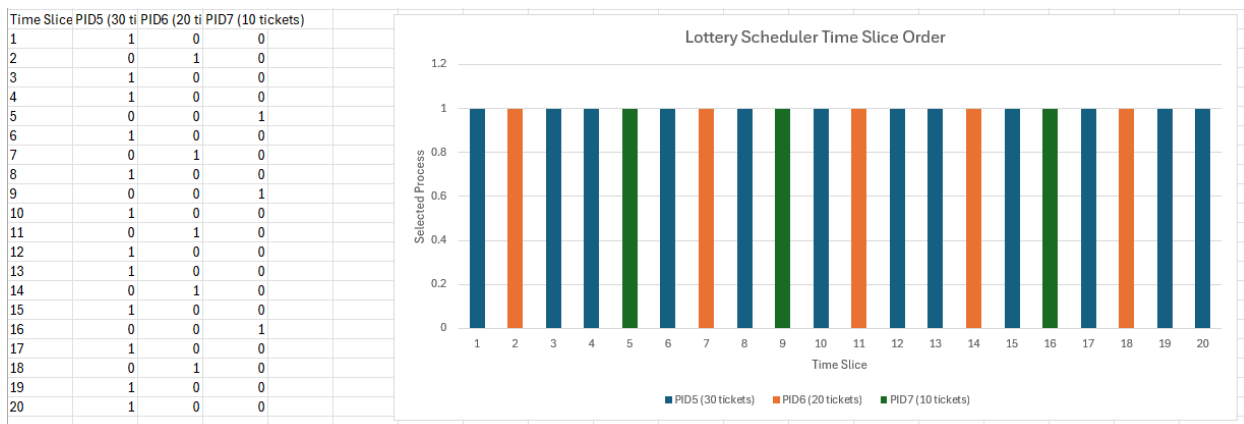
The results show that processes with more tickets received more CPU overall.

CPU Distribution Graph



This graph shows that processes with more tickets were selected more frequently by the scheduler.

Scheduler Time Slice Order



This graph shows the order in which processes were selected during execution and demonstrates randomized scheduling behavior.

Challenges with the Solutions

One challenge I had was registering the new system call correctly, at first the xv6 produced unknown syscall errors because the syscall table entries were missing. I also has to fix Makefile errors because the syscall table entries were missing.

Conclusion

With this project I was successful in implementing a Lottery Scheduler in xv6. Added new process fields, created new system calls, modified the scheduler, and tested the scheduler using multiple processes with different ticket counts. I was able to get a better understanding of an operating system scheduling, process management, and xv6 kernel development.